



# **LAPORAN PRAKTIKUM**

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Muhammad Luthfi Hilmy

NIM : 2428240032

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI  
UNIVERSITAS MULTI DATA PALEMBANG  
TAHUN AKADEMIK 2026/2027**

## Identitas Praktikum

|                         |                       |
|-------------------------|-----------------------|
| Nama Mahasiswa          | Muhammad Luthfi Hilmy |
| NPM                     | 2428240032            |
| Kelas                   | SI5B                  |
| Pertemuan Ke-           | 3                     |
| Topik / Judul Praktikum | Latihan RESTful API   |
| Tanggal Pelaksanaan     | 2026-09-22            |
| Dosen Pengampu          | Nur Rachmat, M.Kom.   |

## I. Tujuan Praktikum

*Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).*

1. Membuat project Express.js
2. Menerima request dari client
3. Membuat backend RESTful API menggunakan Express.js.
4. Membuat routing RESTful
5. Mengirimkan response murni dalam format JSON
6. Mengimplementasikan RESTful API secara utuh (routing, request, response JSON)

## II. Dasar Teori

### A. Environment & Runtime (Node.js, NVM, dan NPM)

Node.js Merupakan *runtime environment* JavaScript yang dibangun di atas mesin V8 milik Google Chrome, memungkinkan JavaScript dijalankan di luar peramban (sisi *server*) serta menjadi fondasi utama bagi *framework* Express.js.

NVM (Node Version Manager) Alat untuk menginstal dan mengelola beberapa versi Node.js sekaligus dalam satu sistem, sehingga memastikan konsistensi versi bagi seluruh tim pengembang.

NPM (Node Package Manager) Manajer paket bawaan Node.js yang digunakan untuk menginstal, memperbarui, dan mengelola dependensi proyek yang dicatat pada berkas package.json dan package-lock.json

## B. Express.js Framework

Express.js adalah *framework* web minimalis dan fleksibel untuk Node.js yang mempermudah pembuatan *server* web dan API tanpa memerlukan banyak kode *boilerplate*. *Fitur utamanya mencakup pemetaan rute (routing) sederhana untuk memetakan URL dan metode HTTP ke fungsi penanganan, dukungan komponen perantara (middleware), serta ekosistem paket pendukung yang luas.*

## C. Arsitektur RESTful API

REST (Representational State Transfer) merupakan gaya arsitektur untuk merancang API berbasis *resource* (sumber daya) yang diakses melalui protokol HTTP.

Prinsip utamanya meliputi:

- a. Resource-Based setiap entitas data (seperti */mahasiswa* atau */tasks*) direpresentasikan melalui URL unik
- b. Stateless setiap permintaan (*request*) bersifat independen, sehingga *server* tidak menyimpan status sesi klien.
- c. Pemetaan HTTP Method ke Operasi CRUD
  - GET: Mengambil/membaca data (*Read*).
  - POST: Menambahkan data baru (*Create*)
  - PUT: Memperbarui data yang ada (*Update*)
  - DELETE: Menghapus data (*Delete*)
- d. Format Data Seragam menggunakan format JSON (*JavaScript Object Notation*) sebagai standar pertukaran data antara klien dan *server*

## D. Anatomi Request, Response, dan Middleware Express.js

- Objek Request ( *req* ) digunakan untuk membaca data masuk dari klien melalui *req.params* (parameter rute dinamis), *req.query* (*query string*), dan *req.body* (data pada *body request*).
  - Middleware *express.json()* Komponen perantara wajib yang mendaftarkan penguraian (*parsing*) JSON sebelum rute agar aplikasi dapat membaca *req.body*.
  - Objek Response ( *res* ) dan Kode Status HTTP digunakan untuk mengirimkan respons kembali ke klien beserta status HTTP yang sesuai. 2xx (Berhasil): 200 OK (berhasil mengambil/mengubah data), 201 Created (berhasil membuat data baru), 204 No Content (berhasil menghapus data). 4xx / 5xx (Gagal/Error): 400 Bad Request, 404 Not Found (data/rute tidak ditemukan), atau 500 Internal Server Error
-

### III. Alat dan Bahan

| No. | Nama Alat / Software | Versi / Keterangan                                                           |
|-----|----------------------|------------------------------------------------------------------------------|
| 1   | Node.js              | menjalankan server aplikasi melalui app.js                                   |
| 2   | Npm                  | mengelola dependensi dan menjalankan script pada package.json                |
| 3   | Express.js           | framework Node.js untuk membuat REST API                                     |
| 4   | dotenv               | membaca konfigurasi dari file .env, seperti nomor port server                |
| 5   | Nodemon              | membantu menjalankan ulang server secara otomatis saat kode berubah          |
| 6   | JavaScript           | bahasa pemrograman yang digunakan pada backend dan frontend                  |
| 7   | HTML                 | membuat halaman pengujian API pada folder frontend-uji                       |
| 8   | Web browser          | membuka halaman HTML dan menguji endpoint API menggunakan fetch()            |
| 9   | REST API/HTTP        | metode komunikasi yang digunakan melalui endpoint GET, POST, PUT, dan DELETE |
| 10  | Visual Studio Code   | editor yang digunakan untuk membuat dan mengelola kode proyek                |

## IV. Langkah Kerja

### 1. Persiapan Environment Node.js & NVM.

Langkah pertama diawali dengan menyiapkan lingkungan pengembangan Node.js menggunakan NVM (Node Version Manager) agar versi yang digunakan dalam tim pengembang tetap konsisten. Pengujian dan instalasi versi LTS dilakukan melalui terminal dengan mengeksekusi perintah `nvm install` *lts*, dilanjutkan dengan memilih versi aktif menggunakan `nvm use` *lts*. Setelah instalasi selesai, kepastian versi Node.js yang sedang berjalan diperiksa dengan menjalankan perintah `node -v`

### 2. Inisialisasi Project Express.js dan Instalasi Dependensi

Langkah berikutnya adalah membuat direktori baru untuk proyek (misalnya `restful-api-app`) dan masuk ke dalamnya melalui terminal. Proyek Node.js kemudian diinisialisasi menggunakan perintah `npm init -y` untuk menghasilkan berkas konfigurasi `package.json`. Selanjutnya, instalasi *framework* Express.js dilakukan dengan menjalankan `npm install express`. Untuk mendukung efisiensi pengembangan, paket Nodemon diinstal sebagai dependensi pengembangan (*devDependency*) melalui perintah `npm install --save-dev nodemon` agar *server* otomatis memuat ulang (*restart*) saat ada perubahan kode. Berkas `package.json` kemudian diperbarui pada bagian `scripts` dengan menambahkan perintah `"dev": "nodemon app.js"`.

### 3. Inisialisasi Server dan Middleware Utama ( `app.js` )

Proses dilanjutkan dengan membuat berkas *entry point* utama aplikasi bernama `app.js`. Di dalam berkas ini, pustaka Express diimpor dan nomor port *server* ditentukan (misalnya `PORT = 3000`). *Middleware* `app.use(express.json())` wajib didaftarkan di bagian atas kode agar *server* Express mampu mengurai (*parsing*) dan membaca data masuk berformat JSON dari `req.body`. Pembuatan *server* diakhiri dengan mendaftarkan *listener* port menggunakan fungsi `app.listen()` agar aplikasi siap menerima permintaan HTTP.

### 4. Perancangan Endpoint dan Penyiapan Data Sementara

Sebelum menuliskan kode logika bisnis, dilakukan perancangan struktur *endpoint* RESTful beserta pemetaan metode HTTP yang sesuai untuk operasi CRUD. Karena belum terhubung ke basis data eksternal, variabel penyimpanan data sementara berbasis memori (*in-memory array*) disiapkan di dalam berkas `app.js` (seperti *array* mahasiswa atau *tasks*) sebagai media manipulasi data.

### 5. Implementasi Routing dan Fungsi Operasi CRUD

Langkah ini berfokus pada pembuatan rute (*routing*) untuk setiap fungsi data. Operasi Read (GET) diimplementasikan melalui `app.get()` untuk mengambil seluruh data serta rute dinamis dengan parameter `req.params.id` untuk mengambil satu data spesifik. Operasi Create (POST) diimplementasikan dengan `app.post()` untuk menerima data baru dari `req.body`, memasukkannya ke *array*, dan mengembalikan respons berstatus 201 Created. Operasi Update (PUT) diimplementasikan melalui `app.put()` berdasarkan ID untuk memperbarui elemen

*array* yang sesuai. Terakhir, operasi Delete (DELETE) diimplementasikan dengan `app.delete()` berdasarkan ID untuk menghapus elemen dari *array* dan mengirimkan respons status 204 No Content.

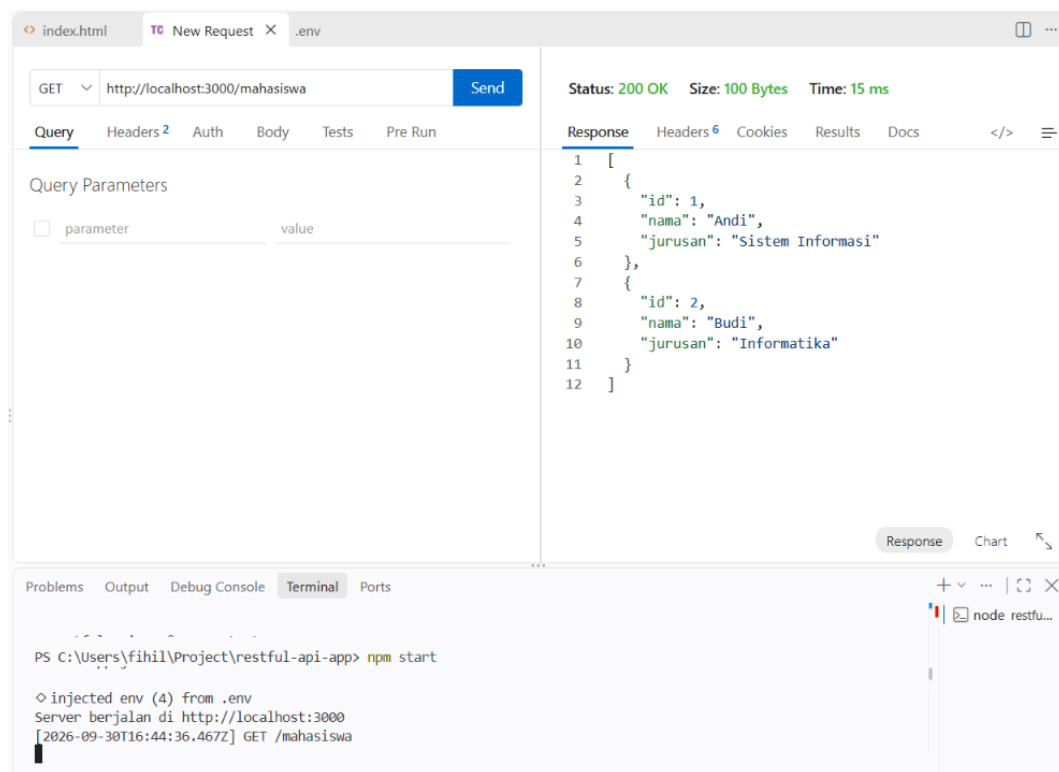
## 6. Pengujian API ( Testing ) dan Dokumentasi Hasil

Langkah terakhir adalah menjalankan *server* aplikasi dengan mengeksekusi perintah `npm run dev` pada terminal. Setiap *endpoint* yang telah dibuat diuji menggunakan alat penguji API (seperti Postman, Thunder Client, atau halaman HTML *frontend* menggunakan fungsi `fetch()`). Seluruh hasil respons JSON dan kode status HTTP dari tiap pengujian didokumentasikan dalam bentuk tangkapan layar (*screenshot*) untuk dilampirkan pada bagian Hasil dan Pembahasan laporan praktikum.

## V. Hasil dan Pembahasan

### 1. Pengujian Endpoint GET /mahasiswa (Read All Data)

Pengujian dilakukan dengan mengirimkan permintaan (*request*) ber-metode GET ke URL `http://localhost:3000/mahasiswa` melalui alat penguji API. *Server* berhasil merespons dengan kode status 200 OK dan mengembalikan data seluruh mahasiswa dalam format *array* JSON.

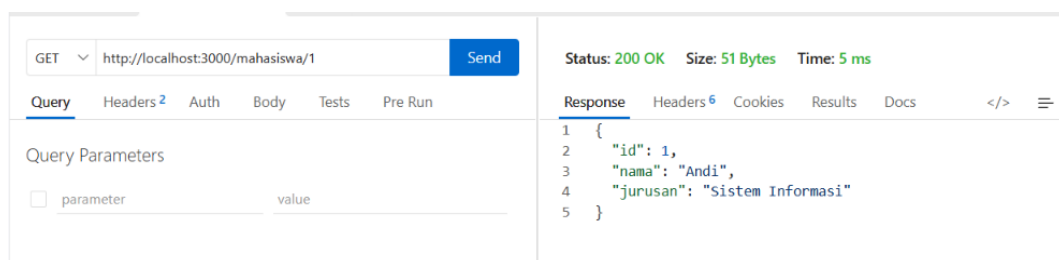


Gambar 1. Hasil Pengujian Endpoint GET /mahasiswa dengan Status 200 OK

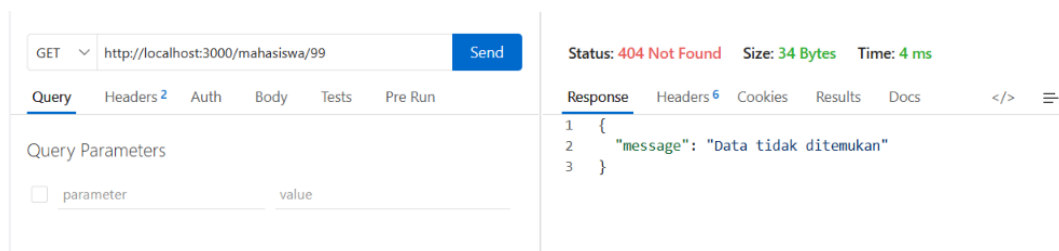
Fungsi `app.get('/mahasiswa', ...)` bertindak sebagai penangan rute untuk mengambil seluruh koleksi data yang tersimpan pada variabel *in-memory array* mahasiswa. Penggunaan kode status 200 OK menandakan bahwa permintaan baca data berhasil diproses oleh *server* dan data dikirimkan pada *body response*.

## 2. Pengujian Endpoint GET /mahasiswa/:id (Read Single Data)

Skenario Berhasil mengirimkan permintaan GET ke `http://localhost:3000/mahasiswa/1` menghasilkan respons status 200 OK beserta objek JSON tunggal berisi data mahasiswa dengan ID 1. Skenario Data Tidak Ditemukan mengirimkan permintaan GET ke `http://localhost:3000/mahasiswa/99` menghasilkan respons status 404 Not Found dengan *body response* `{"message": "Data tidak ditemukan"}`.



Gambar 2. Hasil Pengujian GET /mahasiswa/1 dengan Status 200 OK

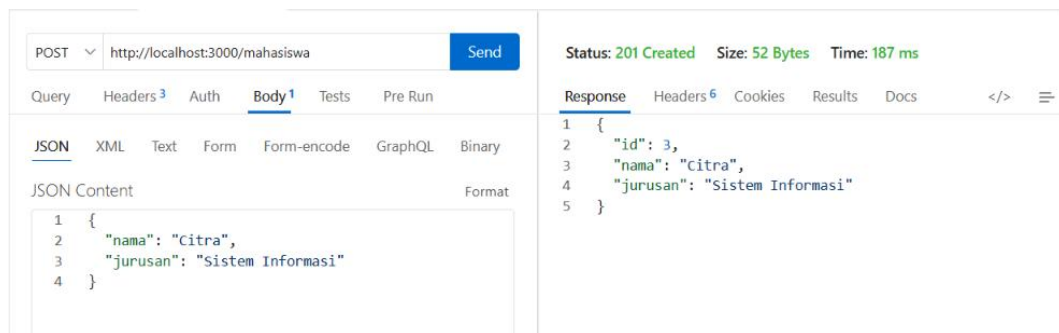


Gambar 3. Hasil Pengujian GET /mahasiswa/99 dengan Status 404 Not Found

Rute ini memanfaatkan parameter rute dinamis `req.params.id` untuk mencari elemen spesifik pada *array* menggunakan fungsi `.find()`. Nilai `req.params.id` perlu dikonversi dari *string* menjadi bilangan bulat (*integer*) menggunakan `parseInt()` agar sesuai dengan tipe data id. Jika data tidak ditemukan, *server* secara eksplisit mengembalikan kode status 404 Not Found sesuai konvensi standar REST API.

## 3. Pengujian Endpoint POST /mahasiswa (Create Data)

Pengujian dilakukan dengan mengirimkan permintaan POST ke `http://localhost:3000/mahasiswa` serta menyertakan *body request* berformat JSON, misalnya `{"nama": "Citra", "jurusan": "Sistem Informasi"}`. *Server* merespons dengan kode status 201 Created dan mengembalikan data objek baru yang telah dilengkapi dengan id otomatis.

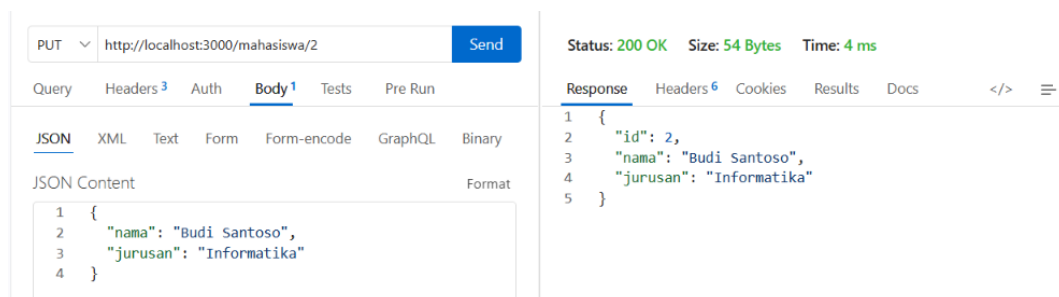


Gambar 4. Hasil Pengujian Endpoint POST /mahasiswa dengan Status 201 Created

Metode POST digunakan untuk menambahkan sumber daya (*resource*) baru ke dalam sistem Data yang dikirim klien dibaca melalui `req.body` berkat dukungan `middleware express.json()` *Server* kemudian membuat ID baru secara dinamis (`mahasiswa.length + 1`), memasukkan objek ke *array*, dan mengembalikan status 201 Created sebagai penanda bahwa data baru berhasil dibuat

#### 4. Pengujian Endpoint PUT /mahasiswa/:id (Update Data)

Mengirimkan permintaan PUT ke `http://localhost:3000/mahasiswa/2` dengan *body request* `{"nama": "Budi Santoso", "jurusan": "Informatika"}` menghasilkan respons 200 OK beserta data mahasiswa yang telah diperbarui.

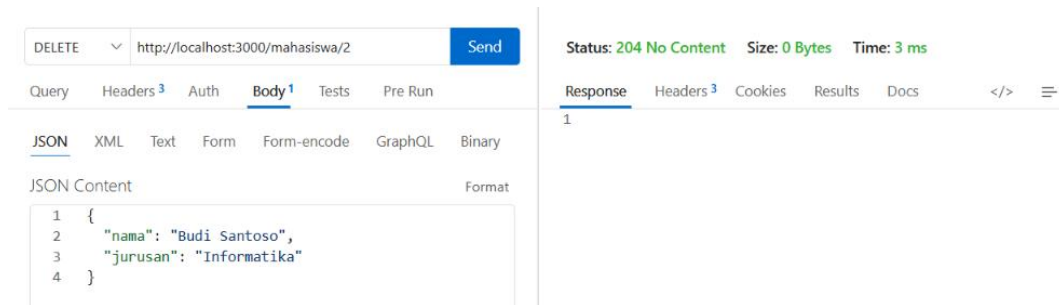


Gambar 5. Hasil Pengujian Endpoint PUT /mahasiswa/2 dengan Status 200 OK

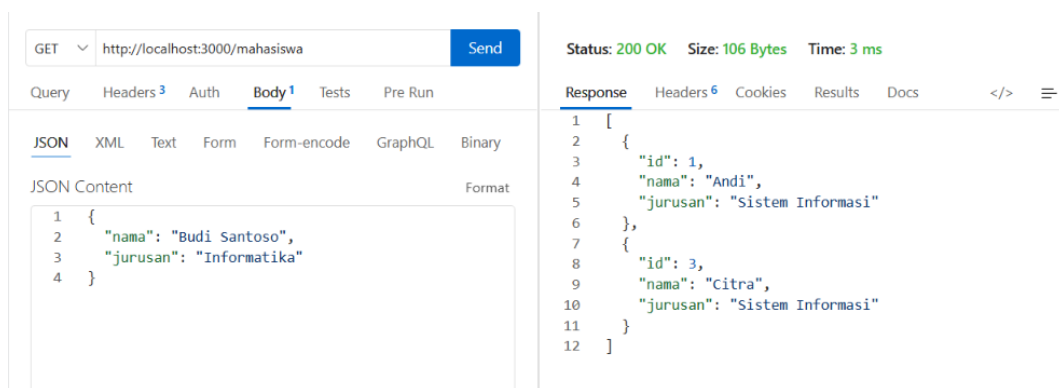
Metode PUT digunakan untuk memperbarui data yang sudah ada berdasarkan parameter ID. *Server* terlebih dahulu mencari indeks data melalui `.findIndex()` Jika ID ditemukan, data lama diperbarui menggabungkan isi `req.body` dan dikembalikan dengan status 200 OK. Sebaliknya, jika ID tidak cocok, *server* mengembalikan respon 404 Not Found.

#### 5. Pengujian Endpoint DELETE /mahasiswa/:id (Delete Data)

Mengirimkan permintaan DELETE ke `http://localhost:3000/mahasiswa/2` menghasilkan respons dengan kode status 204 No Content tanpa membawa *body response*. Pengujian ulang dengan GET /mahasiswa membuktikan bahwa data ber-ID 2 telah terhapus dari *array*.



Gambar 6. Hasil Pengujian Endpoint DELETE /mahasiswa/2 dengan Status 204 No Content



Gambar 7. Verifikasi Data Setelah Pengujian DELETE /mahasiswa/2

Operasi DELETE menghapus elemen dari *array* menggunakan fungsi `.splice()` berdasarkan indeks yang ditemukan. Penggunaan kode status 204 No Content merupakan standar RESTful API untuk mengindikasikan bahwa proses penghapusan berhasil dilaksanakan tanpa perlu mengembalikan konten data tambahan.

## 6. Analisis Penerapan Middleware dan Kode Status HTTP

Seluruh operasi yang melibatkan pembacaan data dari klien (POST dan PUT) sangat bergantung pada *middleware* `app.use(express.json())`. Tanpa pendaftaran *middleware* ini di awal berkas `app.js`, nilai `req.body` akan bernilai `undefined` dan menyebabkan kegagalan proses.

```
// Middleware agar req.body (JSON) dapat dibaca
app.use(express.json());
```

Gambar 8. Penerapan Middleware `express.json()` untuk Membaca Request Body

```
// GET /mahasiswa/:id -> satu data berdasarkan id
app.get('/mahasiswa/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const data = mahasiswa.find((m) => m.id === id);

  if (!data) return res.status(404).json({ message: 'Data tidak ditemukan' });
  res.json(data);
});

// POST /mahasiswa -> tambah data baru
app.post('/mahasiswa', (req, res) => {
  const { nama, jurusan } = req.body;

  if (!nama || !jurusan) {
    return res.status(400).json({ message: 'nama dan jurusan wajib diisi' });
  }

  const baru = { id: nextId++, nama, jurusan };
  mahasiswa.push(baru);
  res.status(201).json(baru);
});
```

Gambar 9. Penerapan Kode Status HTTP pada RESTful API

Penggunaan kode status yang tepat (seperti 200, 201, 204, dan 404) memberikan kejelasan informasi bagi aplikasi klien mengenai status keberhasilan maupun kegagalan setiap *request* yang dikirimkan.

## VI. Kendala dan Solusi

-

## VII. Kesimpulan

Berdasarkan pelaksanaan praktikum mengenai Latihan RESTful API, dapat disimpulkan bahwa:

1. Pembuatan project dan environment proyek *backend* berbasis Express.js berhasil diinisialisasi menggunakan NPM dan dikelola versinya secara konsisten menggunakan Node Version Manager (NVM).
2. Implementasi routing RESTful pengisian dan pemetaan rute (*routing*) berhasil dibangun sesuai prinsip arsitektur REST, yaitu dengan memetakan setiap metode HTTP (GET, POST, PUT, dan DELETE) ke operasi CRUD pada *resource* data.
3. Penerimaan request dan pengolahan data aplikasi mampu menerima dan mengolah berbagai jenis data masukan dari klien melalui req.params untuk parameter rute, req.query untuk *query string*, serta req.body yang diurai menggunakan *middleware* express.json().
4. Penyampaian response JSON dan kode status klien menerima respons murni berformat JSON beserta penyertaan kode status HTTP yang sesuai (200 OK, 201 Created, 204 No Content, dan 404 Not Found) untuk memberikan kejelasan status transaksi data.
5. Capaian pembelajaran praktikum ini secara menyeluruh berhasil memenuhi indikator pembuatan *backend* RESTful API mandiri berbasis Express.js yang siap diintegrasikan dengan aplikasi *frontend* maupun penguji API.

## **Lampiran**

Tautan Repository GitHub:

[https://github.com/fiyoraa/si5b\\_latihan\\_restful\\_MuhammadLuthfiHilmy\\_2428240032](https://github.com/fiyoraa/si5b_latihan_restful_MuhammadLuthfiHilmy_2428240032)

---

Tautan Deployment:

<https://si5b-latihan-restful-muhammad-luthf.vercel.app/>

---